The Matrix Puzzle From Hell, Revisited

2012-10-26 14:33:44

Original: `https://nicknash.me/2012/10/26/happy-halloween/`

1 Dark Origins

This is a short post about a tough puzzle I came across a few months back, originally at Two Guys Arguing (`http://twoguysarguing.wordpress.com/2010/12/30/matrix-puzzle-from-hell-or-how-i-learned-t`
Since it's the Matrix Puzzle From Hell, and it's nearly Halloween, I thought it was ingeniously topical. I'm sure you agree.

If anyone knows the true origin of the puzzle, I'd love to hear it.

2 The Puzzle

A matrix has the property that every entry is the smallest nonnegative integer not appearing either above it in the same column or to its left in the same row.

Give an algorithm to compute the entries of this matrix.

3 An Example

Here's an example matrix:

```
0 1 2 3 4 5 6 7
1 0 3 2 5 4 7 6
2 3 0 1 6 7 4 5
3 2 1 0 7 6 5 4
4 5 6 7 0 1 2 3
5 4 6 7 1 0 3 2
6 7 4 5 2 3 0 1
7 6 5 4 3 2 1 0
```

It looks like this matrix is full of patterns, so what's a good algorithm for computing its entries?

As with any puzzle, it's probably worth trying it before reading the spoilers below. I'll reveal the answer slowly though.

4 A (Very) Naive Algorithm

Without really thinking about the puzzle, we can at least give a correct algorithm. To compute a particular entry, compute all the entries above it, and to its left, and choose the smallest nonnegative integer not included in the union of those two.

If we're considering an $n \times n$ matrix, then to compute a given entry we can be forced to compute up to $\Theta(n^2)$ other entries, because we need to compute all the entries above it and to its left (and all the entries above them and to their left). Once we have the entries above us and to our left we can sort them, in say, $\Theta(n \log n)$ time and then find the smallest excluded element with a linear scan. So the algorithm takes $\Theta(n^3 \log n)$ time.

This certainly feels very naive. We're not using any of the problem's structure. At least we have a time complexity though.

5 Patterns

Let's look at a small example matrix

```
0 1 2 3
1 0 3 2
2 3 0 1
3 2 1 0
```

There's lots of symmetry here, and quite a few patterns to consider. Let's just look at the top-left 2x2 entries though:

```
0 1
1 0
```

To generate the 2x2 sub-matrix of entries in the top-right of our original 4x4 matrix, we just add 2 to these entries. To generate the 2x2 sub-matrix of entries in the bottom-left, we just add 2 to these entries. To generate the 2x2 sub-matrix in the bottom-right, we just copy these entries.

We now have our 4x4 matrix, if we extend it in the same way, adding 4 this time, we'll get our original 8x8 example matrix.

We can even start with a 1x1 matrix: 0, and extend it into the initial 2x2 matrix, by adding one (and copying).

So this looks like a very useful pattern. We've not thought about why, or if, it's right just yet. It looks right though. So let's try and bash out a better algorithm.

6 A Better Algorithm

It looks like we can generate the entire matrix by the "doubling" pattern just described: Start with the 1x1 matrix (which is just 0). Double it to give the 2x2 matrix, double that to give a 4x4 matrix, etc.

To pick out just a single entry, generating the whole matrix isn't going to be the right way to go though. We'd still only be down to $\Theta(n^2)$ time (and space) – not as bad as $\Theta(n^3 \log n)$, but we can do better.

Instead, we can work backwards. To work backwards from a given row and column r, c, we just need to find the smallest number m such that $r, c < 2^m$. Then we know which $2^m \times 2^m$ matrix the entry "belongs" to. Then we find out which quadrant of that matrix the entry is in, and then which quadrant of that quadrant the entry is in, and so on, until we reach the top-left position.

More precisely, we're saying of our matrix M that given r, c:

- Choose the largest integer p such that $2^p \leq r$
- Choose the largest integer q such that $2^q \leq c$
- If $p = q$ (i.e. bottom-right quadrant), then the answer is $M[r - 2^p, c - 2^p]$
- If $p > q$ (i.e. bottom-left quadrant), then the answer is $2^p + M[r - 2^p, c]$
- If $p < q$ (i.e. top-right quadrant), then the answer is $2^q + M[r, c - 2^q]$

Repeating this until we get to $r = c = 0$ only takes $\Theta(\log \max\{r, c\})$ time (because we always more than half either our row or column position), and constant space. That's a **lot** better than our first attempt.

But can we do even better still? It turns out we can. A lot better.

7 Harder, but Easier

When I first heard this puzzle, there was a condition attached to the time complexity of the solution. If it had been that there is a solution working in $O(n^{1/3}(\log n)^2)$ time, I think I would have taken forever trying to think up some complicated algorithm to solve it.

Instead, it said the solution had to work in $O(1)$ time and space per entry. In theory, having to solve a problem in a better time is harder. In the land of puzzles though, *knowing* we can do it in constant time makes getting to an answer easier. It's almost a spoiler.

Knowing it can be done in constant time and space means there must be a formula for the entries. Let's start simple, and go through our tools. Does $M[r, c] = r + c$. No, obviously not. What about the good old bitwise operators though? How about r OR c? Well, no because 1 OR 1 = 1 but we can see $M[1, 1] = 0$ in our examples.

What about XOR? It turns out that's it. We can just XOR our row and column numbers to get the entry. Try it!

8 But Why?

It's a bit of a shocker to know the answer is just plain-old XOR. Who knew it computed the minimum excluded number in a matrix?

Well, if we look at our logarithmic time algorithm, it's pretty easy to see that it's nothing more than an XOR of the row and column. When it said "Choose the largest integer p such that $2^p \leq r$", we're really asking about the most-significant bit of our row number. It's the same with our column, then we compare these bits, in fact, XORing them into our answer, and repeat with the remaining bits.

This at least explains how we get to XOR from our quadrant-based algorithm. We still haven't *really* explained why our initial "doubling" construction was correct though. Before we do that, let's

have a look at a pretty picture.

9 Pretty Picture

As a neat little aside, it turns out that if you draw a picture of our matrix (zero being black), here's what you get:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/10/xor.gif`

Link: `http://nicknash.me/wp-content/uploads/2012/10/xor.gif`

Pretty much any demoscener (or graphics hacker) will recognize this as the XOR texture, the easiest of all textures to generate. Just try googling (`https://www.google.com/search?hl=en&q=XOR+texture`) it!

Speaking of the demoscene, here's video of 4k Amiga intro making extensive use of the XOR pattern (not usually a badge of honor in the demoscene world, incidentally): Humus 4 (`http://www.youtube.com/watch?v=d`

I think it's neat to know that texture is all about minimum excluded numbers, after all these years.

10 But Really, Why?

We never really explained why our doubling construction is actually correct. It's pretty obvious, but let's see about that now.

Suppose we have a $2^k \times 2^k$ version of our matrix M. Our doubling construction gives us a $2^{k+1} \times 2^{k+1}$ matrix, such that:

- The upper-left corner is M
- The bottom-left corner is the entries of M plus 2^k
- The bottom-right corner is M
- The upper-right corner is the entries of M plus 2^k

Now let's think about the simplest version of our matrix. If $k = 0$, we have a 1×1 matrix (containing a single entry, 0) for which every row and every column contains all the numbers in $\{0, \ldots, 2^k - 1\}$. Because of this property of its rows and columns, we'll call that matrix "complete".

If we apply our doubling construction, we have a 2×2 matrix that is also complete. Inductively, it's easy to see we'll continue to get complete power-of-two-sized matrices by doubling complete power-of-two-sized matrices.

The question is now, in a doubled matrix, is each entry the minimum excluded number? Consider any $2^{k+1} \times 2^{k+1}$ matrix of minimum excluded numbers. Take the upper-right corner (**UR**), it can only contain numbers in $\{2^k, \ldots, 2^{k+1} - 1\}$, both in order to guarantee exclusion (because the upper-left corner (**UL**) is complete) and also in order that the entries are as small as possible. Now, if any entry in **UR** was smaller than 2^k plus the corresponding entry in **UL**, then we could subtract 2^k from all the numbers in **UR** and get a $2^k \times 2^k$ matrix of minimum excluded numbers that had a *smaller* entry than **UL** somewhere. This is obviously impossible, because we know **UL** already contains the (unique) minimum excluded numbers in each entry.

We can make similar arguments for the other two quadrants. That's another way to see the XOR connection. The upper-left and bottom-left quadrants are restricted in the numbers they can choose as entries because they have only *either* a different row *or* a different column (i.e. exclusive or) to the entries in the top-left quadrant. Whereas, the entries in the bottom-right quadrant are unrestricted by those in the top-left quadrant having both a different row and column.

11 Nim, Nimbers and Beyond

To finish up, it's worth noting that there's a kid's game played with piles of sticks called Nim (`http://en.wikipedia.org/wiki/Nim`), which the solution to this puzzle represents a winning strategy for.

In fact, there are even things called Nimbers (`http://en.wikipedia.org/wiki/Nimber`), and XOR corresponds to Nimber addition. There's also a Nimber multiplication operator. I wonder what it looks like as a texture, compared to what the XOR texture looks like?